



西安交通大学
XI'AN JIAOTONG UNIVERSITY

面向边缘智能应用的低功耗 Int8 数字存算一体架构设计及 FPGA 验证

学 号： 2221411116

姓 名： 焦宏宝

导 师： 梁峰

论文题目： 面向边缘智能应用的低功耗 Int8 –
数字存算一体架构设计及 FPGA 验证

专 业： 微电子科学与工程强芯实验班

学 院： 电信学部 微电子学院

填写时间： 2026 年 04 月

摘 要

随着深度学习在边缘智能场景中的广泛部署，传统冯·诺依曼架构中处理器与存储器分离所带来的数据搬运瓶颈日益突出。存算一体（Computing-in-Memory, CIM）技术通过在存储阵列内部或近存储位置完成计算，从体系结构层面缓解“内存墙”问题，被认为是提升推理能效与吞吐率的重要方向。

本文设计并实现了一种面向边缘推理的低功耗 Int8 数字存算一体矩阵计算阵列架构。系统以 16×16 CIM Tile 为原子计算单元，采用参数化设计支持可配置并行度，核心引擎通过 7 级流水线状态机实现权重加载、零点减法、MAC 运算、偏置累加、ReLU 激活、重量化与结果写回的完整推理数据通路。权重存储采用 16-bank SRAM 拆分架构以确保 Vivado BRAM 推断，并通过 AXI4-Lite 接口实现 DMA 式权重写入与 CSR 运行时配置。

在软核控制方面，集成了 PicoRV32 RISC-V 软核替代 ARM PS 进行推理控制，通过 Wishbone-to-CSR 桥接实现与 CIM IP 完全相同的 AXI 事务，配合双端口 BRAM 实现运行时固件加载与结果回读，使 CIM 加速器可在纯 PL 侧自主运行。

本设计在 PYNQ-Z2 (Zynq-7020) 平台完成综合部署与板级验证。ARM 控制模式下，MLP 网络 ($784 \rightarrow 128 \rightarrow 10$) 以 60MHz 工作频率实现单幅推理 3282 周期，LeNet-5 CNN (7 层) 实现 76034 周期，20 张 MNIST 手写数字图片上分类准确率 100% (模型在完整测试集上约 93%)，200 张图片上准确率 99.5%，全部输出与 Python bit-accurate 参考模型逐元素一致。PicoRV32 模式以 50MHz 时序收敛，增加仅 1627 个 LUT 与 9 个 BRAM 的开销，200 张 MNIST 图片验证准确率 96.5%，全部 logits 与 ARM 控制路径 bit-exact 一致。相对 ARM Cortex-A9 纯 NumPy 基线，MLP 推理加速比约 25.8 倍，LeNet-5 加速比约 20.9 倍 (sequential 逐像素模式对比)。系统占用 LUT 20.84%、BRAM 25%、DSP 100% (220 个)，CIM IP 动态功耗仅 0.284W。

关键词: 存算一体; INT8 量化推理; FPGA 验证; CIM 阵列; PicoRV32; PYNQ-Z2

Abstract

With the widespread deployment of deep learning at the edge, the data movement bottleneck caused by the separation of processor and memory in traditional von Neumann architectures has become increasingly prominent. Computing-in-Memory (CIM) technology mitigates the "memory wall" problem at the architectural level by performing computation within or near memory arrays, and is considered a promising direction for improving inference energy efficiency and throughput.

This thesis designs and implements a low-power INT8 digital computing-in-memory matrix computation array architecture for edge inference. The system employs a 16×16 CIM Tile as the atomic compute unit with parameterized parallelism. The core engine implements a 7-stage pipelined FSM covering the complete inference datapath: weight loading, zero-point subtraction, MAC operation, bias accumulation, ReLU activation, requantization, and result writeback. Weight storage uses a 16-bank SRAM split architecture for reliable Vivado BRAM inference, with an AXI4-Lite interface for DMA-style weight loading and runtime CSR configuration.

For soft-core control, a PicoRV32 RISC-V core is integrated to replace ARM PS for inference control, using a Wishbone-to-CSR bridge that issues the same AXI transactions as the ARM path, with dual-port BRAMs for runtime firmware loading and result readback.

The design is synthesized and verified on a PYNQ-Z2 (Zynq-7020) platform. Under ARM control at 60MHz, the MLP network ($784 \rightarrow 128 \rightarrow 10$) achieves 3282 cycles per inference, and LeNet-5 CNN (7 layers) achieves 76034 cycles, achieving 100% classification accuracy on 20 test images (model accuracy $\sim 93\%$ on the full MNIST test set) and 99.5% on 200 test images respectively, all outputs bit-exact with the Python golden model. The PicoRV32 mode achieves timing closure at 50MHz with only 1627 additional LUTs and 9 BRAMs, validated on 200 MNIST images with 96.5% accuracy and 100% bit-exact logits. Compared to a NumPy baseline on ARM Cortex-A9, the CIM hardware achieves approximately $25.8\times$ speedup for MLP and $20.9\times$ for LeNet-5 (sequential per-pixel comparison). The system utilizes 20.84% LUTs, 25% BRAM, 100% DSP (220 units), with the CIM IP consuming only 0.284W dynamic power.

Keywords: Computing-in-Memory; INT8 quantized inference; FPGA verification; CIM array; PicoRV32; PYNQ-Z2

目录

1 绪论	6
1.1 选题背景	6
1.2 国内外现状及发展趋势	7
1.3 本文主要工作与贡献	7
2 INT8 CIM 阵列架构设计	8
2.1 整体架构	8
2.2 CIM Tile 设计	9
2.3 存储子系统	10
2.3.1 权重 SRAM 的 BRAM 推断问题与 bank 拆分方案	10
2.3.2 偏置 SRAM 与输入缓冲	10
2.4 量化与重定标	11
2.5 AXI4-Lite 接口	11
2.6 7 级流水线状态机	12
3 PicoRV32 RISC-V 软核集成	13
3.1 设计动机	13
3.2 系统架构	13
3.3 Bus Bridge 与地址映射	14
3.4 模型规模约束	14
4 FPGA 验证与性能评估	15
4.1 验证方法学	15
4.2 PYNQ-Z2 综合结果	15
4.3 MLP 网络推理验证	16
4.4 LeNet-5 CNN 推理验证	17
4.5 加速比分析	17
4.5.1 理论分析	17
4.5.2 硬件推理时间	18
4.5.3 软件基线与加速比	18
4.6 PicoRV32 验证结果	19
4.6.1 PicoRV32 功能验证	19
4.7 两种控制模式对比	20
5 总结与展望	20
5.1 工作总结	20
5.2 不足与展望	21

1 绪论

1.1 选题背景

近年来,以 YOLO^[1]、SSD^[2]为代表的深度学习目标检测算法在智能安防、工业视觉、无人系统与车载感知等场景中被广泛采用,其共同特点是需要较高精度下实现接近实时的推理能力,而网络内部又以卷积/矩阵乘加(MAC)为主导计算负载,导致整体计算量与数据访问量都十分庞大。YOLO 将检测任务端到端地建模为单网络回归问题,SSD 通过多尺度特征图上的卷积预测实现单阶段检测,这类方法在追求速度与精度的同时,也使得卷积算子成为系统性能与功耗优化的核心瓶颈之一。

然而,在传统冯·诺依曼架构中,处理器与存储器相互分离,权重与特征图等数据需要在存储层级与计算单元之间频繁搬运,系统性能逐渐受限于存储带宽与访问延迟,“内存墙(Memory Wall)”问题由此形成并长期存在。Wulf 与 McKee 早在 1995 年就指出处理器速度提升与存储器速度提升之间的差距会不断扩大,从而限制系统级性能提升^[3]。与此同时,能效维度的矛盾更加突出:在先进工艺条件下,数据搬运(尤其是跨层级、跨片外存储访问)相对于算术运算存在显著更高的能量代价,使得“算得快”并不等价于“算得省”。Horowitz 在 ISSCC 2014 的报告中系统讨论了计算系统的能耗拆分,强调需要从体系结构与数据流层面减少昂贵的数据访问^[4]。面向 CNN 的专用加速器研究也进一步表明:通过数据复用与数据流优化降低(尤其是片外)访存开销,是实现低功耗实时推理的关键路径之一^[5]。在边缘计算场景(电池供电、散热受限、实时性要求高)中,上述矛盾更加尖锐,传统“分离式计算-存储”模式难以同时满足吞吐与能效需求。

存算一体(Computing-in-Memory, CIM)技术通过在存储阵列内部或近存储位置完成计算,将“数据搬运”转化为“原位计算”,从体系结构层面缓解冯·诺依曼瓶颈,被认为是突破内存墙、提升能效与吞吐率的重要方向。Nature Electronics 的综述工作提出了覆盖不同存储器参与程度的 CIM 全谱系分类框架,并指出 CIM 可在逻辑运算与乘加运算等基本原语上实现能效提升^[6]。在诸多实现路径中,基于标准 CMOS 工艺的 SRAM-CIM 具有速度快、工艺成熟、可靠性高、与主流数字设计流程兼容等优点,是面向工程落地与系统集成的重要路线之一^[7]。

从应用角度看,为在精度、硬件复杂度与系统吞吐之间取得平衡,Int8 量化推理已成为工业界与学术界常用的落地方案之一。Jacob 等人提出的整数算术推理量化方法表明,通过合理的量化尺度与训练/校准策略,可在保持精度的前提下实现高效的整数运算推理流程^[8]。从硬件角度看,已有研究在 SRAM-CIM 宏单元中展示了面向多比特 MAC 的实现范式,如 8b MAC 的 SRAM CIM 宏以及支持 Signed-INT8 的 SRAM CIM 宏等工作,说明“多比特、可用于实际推理精度需求”的 SRAM-CIM 在工程上具有可行性^[9]。

基于上述背景,本课题面向边缘智能推理的能效与实时性需求,在标准

CMOS SRAM 工艺假设下,设计了一种支持 Int8 数据类型的数字存算一体矩阵计算阵列架构:在存储阵列内实现权重存储与乘法/部分积生成,结合合理的阵列并行组织、累加与数据通路调度,降低数据搬运开销并提升并行吞吐;同时采用 SystemVerilog 完成 RTL 级实现,并在 FPGA 平台完成综合部署与功能验证,形成了可复现的验证平台与完整代码交付。

1.2 国内外现状及发展趋势

存算一体研究近年来快速发展,已形成覆盖器件/电路/架构/算法的多层次技术体系。相关综述工作从实现介质 (SRAM、DRAM、RRAM/PCM 等) 与计算形态 (模拟/数字、阵列内/近存储) 对 CIM 进行系统分类,指出其共同目标是减少访存与数据移动开销、利用阵列并行提升吞吐^{[10][7][11][12]}。

在国际研究中,基于新型非易失存储器 (如 RRAM) 交叉阵列的模拟 CIM 方案曾是重要热点,典型代表包括 ISAAC^[13]和 PRIME^[14]等工作。PUMA 等可编程 memristor 加速器进一步探索了更可编程的推理加速路径^[15]。但模拟 CIM 往往需要 ADC/DAC 等外围电路并面临器件非理想性、噪声、精度与可校准性等挑战^{[6][16]}。面向通用存储体系的近存储计算也形成了多条路线,如 Ambit^[17]、Neural Cache^[18]、FloatPIM^[19]等。

在电路与工程实现层面,基于标准 CMOS 工艺的 SRAM-CIM 因其速度、成熟度与数字设计流程兼容性而成为当前研究与产业化的重要路线^{[7][11][12]}。已有多项芯片级宏单元工作给出了可参考的实现范式,如 28nm 工艺下实现 8b MAC 的 SRAM-CIM 宏单元,以及支持 Signed-INT8 的 ADC-less SRAM-CIM 宏单元^[9]。研究趋势正从“单个 CIM 宏单元验证”走向“可重构/可编程的数字 CIM 处理器与系统级加速器”。Tu 等报道了支持 BF16 与 INT8 的可重构数字 CIM 处理器^[20],其后续 ReDCIM 工作给出了统一 FP/INT 的数字 CIM 设计^[21]。同期也出现面向 Transformer/注意力工作负载的数字 CIM 加速研究,如 TranCIM^[22]、MulTCIM^[23]等。

在算法侧,Int8 量化推理已成为常用的落地路径。Jacob 等工作提出的整数算术推理量化方法提供了硬件采用 Int8 数据通路的算法依据^[8]。PACT^[24]与 LSQ^[25]等研究进一步从训练与量化参数学习角度改善量化精度。

综合来看,国内外 CIM 研究呈现如下趋势:技术路线从“模拟/新型器件主导”扩展为“模拟与全数字并行发展”;算子与精度从低比特走向多比特 MAC (尤其是 Signed-INT8);应用从 CNN 拓展到 Transformer/多模态;验证方式越来越需要体系结构层面的可复现评估 (包括 RTL/FPGA 原型验证、资源/频率/吞吐/延迟评估等)^{[11][26]}。本课题正是在此背景下,以数字 SRAM-CIM 阵列架构为核心,围绕数据通路、控制调度与 FPGA 验证建立了工程闭环。

1.3 本文主要工作与贡献

本文完成了以下工作:

- (1) 设计并实现了参数化的 16×16 INT8 CIM Tile 阵列与 7 级流水线加速器核心，支持可配置并行度 (PAR_OB 参数)，在单一 FSM 引擎中处理任意层的 MVM 运算，避免了为每一层设计专用硬件的冗余。
- (2) 解决了权重 SRAM 的 BRAM 推断问题：将 2048 位宽的 weight SRAM 拆分为 16 个独立的 128 位 bank，确保 Vivado 成功推断为 BRAM；设计了 read-modify-write 式的 AXI DMA 写入通路，在保持 32 位 AXI 接口的同时实现 128 位整行写入。
- (3) 集成 PicoRV32 RISC-V 软核替代 ARM PS 进行推理控制，设计了 Wishbone-to-CSR 桥接、双端口 FW/Result BRAM 等模块，实现了纯 PL 侧自主推理能力。
- (4) 建立了”Python golden model $\rightarrow$ VCS 仿真 $\rightarrow$ PYNQ 上板”三层一致的验证方法学，在 PYNQ-Z2 平台上完成了 MLP (20 张图片 100% 准确率) 和 LeNet-5 CNN (200 张图片 99.5% 准确率) 的端到端验证，全部输出与软件参考模型 bit-exact 一致。
- (5) 提供了完整的性能评估数据：资源占用、时序、周期数、功耗与加速比分析，形成了可复现的工程交付物。

2 INT8 CIM 阵列架构设计

2.1 整体架构

本设计采用”输入缓冲—CIM 阵列—累加与量化—输出缓冲”的流水式组织，核心引擎以有限状态机 (FSM) 驱动完整的推理数据通路。图1展示了系统整体架构。

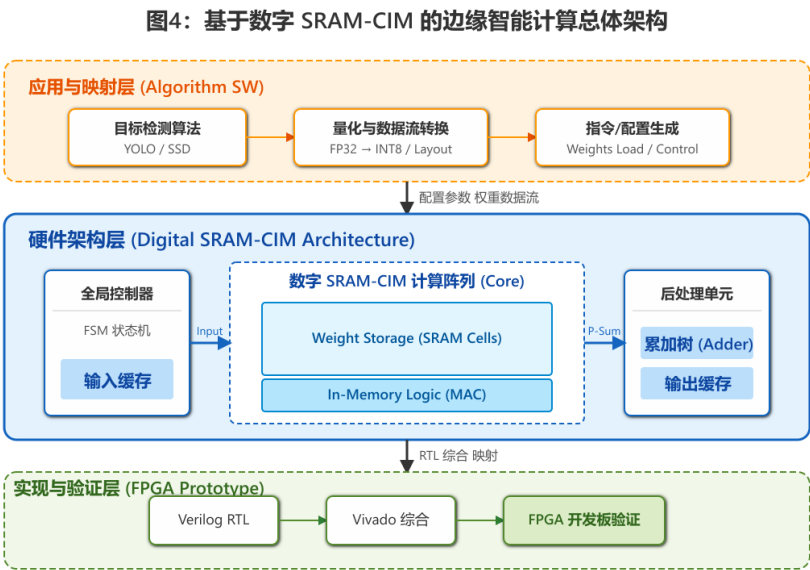


图 1: CIM SoC 整体架构

系统由以下关键模块组成：

- **cim_accel_core**: 核心加速器引擎, 包含 17 状态的 FSM 与 7 级流水线, 负责计算调度与性能计数;
- **cim_tile**: 16×16 CIM 原子计算单元, 纯组合逻辑, 实现 256 个 INT8 MAC 运算;
- **psum_accum**: 部分和累加器, 带优先级清零机制;
- **weight_sram**: 16-bank 权重 SRAM, 支持 AXI DMA 写入与 tile-packed 读取;
- **bias_sram**: 偏置 SRAM, 按输出神经元索引寻址;
- **input_buffer**: 输入缓冲, 支持 AXI 写入与零点减法;
- **output_buffer**: 输出缓冲, 含 argmax 逻辑;
- **cim_axi_lite_slave**: AXI4-Lite 接口, 提供 CSR 配置与存储窗口访问。

所有可配置参数集中管理于 `cim_pkg.sv` 包中, 包括 Tile 尺寸 (TILE_ROWS=16, TILE_COLS=16)、并行度 (PAR_OB)、数据位宽 (INT8 输入/权重、INT32 偏置/累加器)、最大层维度 (MAX_IN_DIM=784, MAX_OUT_DIM=128)、CSR 地址映射与 FSM 状态编码。全局无魔法数字, 所有 RTL 模块通过 `import cim_pkg::*` 引用参数。

加速器的工作流程为: PS (或 PicoRV32) 通过 AXI 接口写入权重、偏置与输入数据, 配置层参数 (IN_DIM, OUT_DIM, N_IB, N_OB, requant_mult, requant_shift, act_mode, input_zp), 然后触发 start 信号。核心引擎按 output-block 并行、input-block 迭代的顺序完成 MVM 运算, 每轮计算后进行偏置累加、激活与重量化, 最终将 INT8 结果写入输出缓冲。完成后置位 done 信号并可触发 IRQ。

2.2 CIM Tile 设计

CIM Tile 是本设计的原子计算单元, 实现以下运算:

$$\text{psum}[r] = \sum_{c=0}^{15} \text{x_eff}[c] \times w[r][c], \quad r = 0, 1, \dots, 15 \quad (1)$$

其中 `x_eff` 为经过零点减法后的 9 位无符号有效输入, `w` 为 Signed-INT8 权重, 输出为 32 位有符号部分和。原始 UINT8 像素值 `[0, 255]` 减去零点 `input_zp = -128` (即加上 128), 得到范围为 `[128, 383]` 的 9 位无符号值。

Tile 采用纯组合逻辑实现, 通过 `generate` 构造 16 行 × 16 列的乘加阵列。每行独立计算一个输出神经元的部分和, 列方向采用链式累加:

$$\text{row_acc}[c + 1] = \text{row_acc}[c] + \text{signed_extend}(\text{x_eff}[c]) \times w[r][c] \quad (2)$$

每个 Tile 每个时钟周期产生 $16 \times 16 = 256$ 个 MAC 运算的结果。采用纯组合设计的原因是将时序控制集中在核心引擎的 FSM 中，Tile 本身作为”存储阵列做计算”的数字抽象。

2.3 存储子系统

2.3.1 权重 SRAM 的 BRAM 推断问题与 bank 拆分方案

权重 SRAM 的设计是本项目的关键工程挑战之一。初始设计中，weight SRAM 为一整块 2048 位宽 ($\text{TILE_ROWS} \times \text{TILE_COLS} \times 8 = 2048$ 位) 的存储器。然而，Vivado 对这种超宽 memory 推断 BRAM 本身就困难；更关键的是，如果用 bit-select 方式做 32 位 chunk 部分写入，Vivado 会直接放弃 BRAM 推断，退化为纯寄存器，导致资源爆炸。

解决方案是将 weight SRAM 拆分为 16 个独立的 bank，每个 bank 仅 $\text{ROW_W} = \text{TILE_COLS} \times \text{WEIGHT_W} = 128$ 位宽，对应 Tile 的一行权重。16 个 bank 通过 generate 实例化，每个 bank 独立推断为 BRAM：

$$\text{bank_mem}[r][\text{tile_idx}] = \{w[r][15], w[r][14], \dots, w[r][0]\}, \quad r = 0, \dots, 15 \quad (3)$$

写入侧，AXI slave 使用 staging register 将 4 个 32 位 AXI 写入累积为完整的 128 位行数据，然后通过单周期 whole-word 写入送入对应 bank。这确保了 BRAM 推断所需的”纯整字写入”条件。

表 1: 权重 SRAM 设计参数

参数	值	说明
Bank 数量	16	对应 TILE_ROWS
Bank 宽度	128 bits	$\text{TILE_COLS} \times \text{WEIGHT_W}$
Bank 深度	392	$\text{MAX_N_OB} \times \text{MAX_N_IB} = 8 \times 49$
总容量	100,352 bytes	$16 \times 128 \times 392 / 8$
写入接口	128-bit whole-word	确保 BRAM 推断
读取延迟	1 cycle	registered read

2.3.2 偏置 SRAM 与输入缓冲

偏置 SRAM 按输出神经元索引寻址，每个条目 32 位 (INT32)，深度为 $\text{MAX_OUT_DIM}=128$ 。输入缓冲存储 tile-packed 的输入数据，每条目 $\text{TILE_COLS} \times$

INPUT_W = 128 位，深度为 MAX_N_IB=49。输入缓冲在读取时自动完成零点减法：

$$x_eff[c] = input[c] - input_zp \quad (4)$$

其中 input_zp 通过 CSR 配置，默认为 -128（相当于将 UINT8 像素值整体上移 128，确保 x_eff 为非负的 9 位无符号值，范围 [128, 383]）。

2.4 量化与重定标

本设计的量化流程遵循标准 INT8 整数推理规范^[8]，确保与软件 golden model 的 bit-exact 一致性。完整的量化链路为：

$$psum = \sum_c x_eff[c] \times w[r][c] + bias[r] \quad (5)$$

$$prod = psum \times requant_mult \quad (6)$$

$$shifted = \left\lfloor \frac{prod + 2^{requant_shift-1}}{2^{requant_shift}} \right\rfloor \quad (7)$$

$$output = clamp(shifted, -128, 127) \quad (8)$$

其中 requant_mult 和 requant_shift 为每层可配置的量化参数，通过 CSR 在运行时设置。重定标过程中使用 64 位中间乘积以避免溢出，移位时采用带舍入的算术右移（round-half-up），最后 clamp 到 INT8 范围 [-128, 127]。

激活函数支持三种模式：无激活（ACT_NONE）、ReLU（ACT_RELU，将负值归零）、范围截断（ACT_CLAMP）。模式通过 CSR 的 ACT_MODE 寄存器配置。

2.5 AXI4-Lite 接口

系统通过 AXI4-Lite slave 接口与外部控制器（PS ARM 或 PicoRV32）交互，采用 14 位地址空间（16KB），划分为以下区域：

表 2: CSR 地址映射

地址范围	偏移	功能
0x000–0x00C		控制/状态/IRQ 寄存器
0x010–0x02C		层配置 (IN_DIM, OUT_DIM, N_IB, N_OB, 量化参数)
0x030–0x03C		性能计数器 (cycle count, MAC count)
0x040		argmax 预测结果
0x044–0x04C		权重 DMA 控制 (地址/数据/控制)
0x100–0x2FF		Logits 回读窗口
0x1000–0x1FFF		输入缓冲写入窗口
0x2000–0x2FFF		偏置缓冲写入窗口

权重写入采用 DMA 式接口: 软件设置 CSR_WDMA_ADDR (tile 索引) 与 CSR_WDMA_DATA (32 位数据块), 通过 CSR_WDMA_CTRL 的 chunk_idx 字段指定当前写入的是 128 位行的第几个 32 位块。AXI slave 内部使用 staging register 累积 4 个 chunk 后, 生成一次 128 位 whole-word 写入到 weight_sram。

2.6 7 级流水线状态机

核心引擎的 FSM 包含 17 个状态, 其中计算数据通路划分为 7 级流水线, 分为两个流水区域:

计算流水线 (每个 input-block 迭代):

1. **ST_FETCH / ST_WAIT_SRAM**: 发出 BRAM 读地址, 等待权重 SRAM 输出 (1 cycle latency);
2. **ST_XEFF_REG**: 输入 BRAM 输出就绪 + 零点减法 → 锁存 x_eff_reg。
关键路径: BRAM T_{co} + 32 位减法 + clamp (约 10ns);
3. **ST_MAC**: cim_tile 计算 $x_eff_reg \times w_tile_reg$ → 锁存 tile_psum_reg。
关键路径: 16 元素 MAC 链 (约 10ns);
4. **ST_COMPUTE**: $psum_accum += tile_psum_reg$ (寄存器加法)。关键路径: 32 位加法 (约 4ns)。

输出流水线 (每个 output neuron, 所有 IB 迭代完成后):

5. **ST_BIAS_ADD** → **ST_ACTIVATE**: 加偏置 → ReLU 激活;
6. **ST_STORE**: 64 位乘法 ($psum \times requant_mult$) → 锁存 prod_r;
7. **ST_SHIFT_CLAMP** → **ST_WRITE_OBUF**: 移位 + 舍入 + clamp → 写输出缓冲。

将原本的 26ns 组合路径拆分为多级流水后，每级关键路径控制在约 10ns 以内。代价是每个 IB 迭代增加 2 个额外周期 (XEFF_REG + MAC)，对于 FC1 层 (49 个 IB 迭代)，计算周期从约 392 增加到约 539。这一权衡使得设计可以在更高频率下工作。

3 PicoRV32 RISC-V 软核集成

3.1 设计动机

Step 1-3 中的 CIM 加速器由 ARM PS 通过 Python/MMIO 控制。然而，项目任务指导书要求实现软核设计，因此不应该直接使用 ARM PS，而需要在 PL 侧实现自主控制能力。本设计选择集成 PicoRV32 开源 RISC-V 软核 (YosysHQ, MIT License)，其优势包括：

- 单文件实现 (picorv32.v)，RV32IM 指令集；
- 资源开销低：约 1500 LUT；
- 成熟的 native memory interface，适合直接桥接到 CIM CSR。

最终目标是让 CIM 加速器在纯 PL 中自主运行推理，不依赖 PS 控制。

3.2 系统架构

PicoRV32 集成后的系统架构如图2所示。

```

PicoRV32 --bridge--- FW BRAM (32KB, 代码+数据)
                  +--- CIM IP (via mini AXI master)
                  +--- UART TX (debug)
                  +--- Result BRAM (256B, 推理结果)

PS (ARM) --AXI----- FW BRAM port B (写入 firmware)
                  +--- Result BRAM port B (读取结果)
                  +--- AXI GPIO (控制 cpu_rst_n)

```

图 2: PicoRV32 + CIM SoC 架构

设计要点包括：

(1) **CIM IP 不做修改**：PicoRV32 通过 mini AXI master 发送与 ARM 完全相同的 AXI 事务，CIM IP 的 AXI slave 接口无需任何改动，确保了与 Step 2 硬件的完全兼容。

(2) **双端口 FW BRAM**：固件通过 PS 侧 AXI 写入 FW BRAM port B，而不是烧写到 bitstream 中。这使得每次更换测试图片无需重新生成 bitstream，

只需重新加载 firmware.hex。FW BRAM 容量 32KB，足以容纳 784→16→10 小型 MLP 模型的权重数据与 C 固件代码（约 14.5KB）。

(3) **Result BRAM**: PicoRV32 将推理结果（预测类别、logits、magic 标记）写入 Result BRAM，PS 通过 AXI 读取结果。这避免了 PL 侧 UART 与 PS 侧 USB 的连接性问题。

(4) **CPU 复位控制**: PS 通过 AXI GPIO 控制 cpu_rst_n 信号，实现”hold → write firmware → release”的操作流程。

3.3 Bus Bridge 与地址映射

picorv32_cim_bridge 模块将 PicoRV32 的 native memory interface 桥接到各外设，地址映射如下：

表 3: PicoRV32 地址映射

地址范围	大小	设备
0x0000_0000 – 0x0000_7FFF	32KB	FW BRAM (port A)
0x4000_0000 – 0x4000_3FFF	16KB	CIM CSR / 存储窗口
0x8000_0000	4B	UART TX 数据寄存器
0x8000_0004	4B	UART TX 状态寄存器
0xC000_0000 – 0xC000_00FF	256B	Result BRAM (port A)

Bridge 根据地址高位进行译码，对 CIM CSR 区域的访问转换为 AXI 读写事务。PicoRV32 访问 CIM 的流程与 ARM 通过 MMIO 访问完全等价，固件中的 CSR 读写函数直接对应 AXI 事务。

3.4 模型规模约束

由于 PicoRV32 额外占用 LUT 资源，且固件需要放入 32KB FW BRAM，模型规模受到限制。FW BRAM 中需要存储：C 固件代码、权重数据（INT8 打包）、偏置数据（INT32）与输入数据。对于 784→16→10 的小型 MLP 模型，数据量约为：

$$784 \times 16 \times 1 + 16 \times 10 \times 1 + (16 + 10) \times 4 + 784 \times 1 \approx 14.5\text{KB} \quad (9)$$

加上 C 固件代码约 4KB，总计约 18.5KB，可以放入 32KB BRAM 中。该模型在 MNIST 测试集上 INT8 量化推理准确率超过 90%。

4 FPGA 验证与性能评估

4.1 验证方法学

本项目建立了”Python golden model → VCS 仿真 → PYNQ 上板” 三层一致的验证方法学：

第一层：Python bit-accurate 参考模型。 golden_model.py 以纯整数算术实现与硬件完全相同的计算流程（零点减法、MVM、偏置累加、ReLU、重量化），输出 hex 格式测试向量供 RTL 仿真和板级验证使用。

第二层：VCS RTL 仿真。 采用”单元级 → 阵列级 → 系统级” 分层验证策略：

- **tb_cim_tile**: CIM Tile 单元测试，103 个随机向量，验证 MAC 运算正确性；
- **tb_cim_accel_core**: 系统级 MVM 测试，覆盖随机向量与边界用例（全零、全 -128、最大维度等）；
- **tb_mnist_e2e**: MNIST 端到端测试，784→128→10 两层 MLP，与 golden model 逐元素对比。

通过 run_regression.sh 脚本一键运行全部测试并汇总 PASS/FAIL 结果。

第三层：PYNQ-Z2 上板验证。 将 bitstream 与 hwh 文件部署到 PYNQ 板，通过 Jupyter Notebook 运行 Python 驱动程序（cim_driver.py 的 CIMModel 类），加载真实 MNIST 手写数字图片，逐层执行推理并与 golden model 对比。

4.2 PYNQ-Z2 综合结果

设计在 Vivado 2024.2 中针对 Zynq-7020 (xc7z020clg400-1) 进行综合与实现。以下为 ARM 控制模式（不含 PicoRV32）的综合结果。

表 4: CIM SoC 资源占用（ARM 控制模式，PAR_OB=1, PYNQ-Z2）

资源类型	使用量	可用量	占用率
Slice LUT	11,087	53,200	20.84%
LUT as Logic	11,027	53,200	20.73%
LUT as Memory	60	17,400	0.34%
Slice Register (FF)	5,385	106,400	5.06%
Block RAM (36Kb)	34	140	24.29%
Block RAM (18Kb)	2	280	0.71%
DSP48E1	220	220	100.00%

DSP 占用率达到 100% 是因为 PAR_OB=1 时，CIM Tile 的 $16 \times 16 = 256$ 个乘法器中的大部分映射到了 DSP48E1。Zynq-7020 共有 220 个 DSP48，恰好被

完全利用。如果需要降低 DSP 占用，可以通过 Vivado 综合属性将部分乘法器映射到 LUT。

时序方面，时钟约束为 60MHz (16.7ns 周期)，布局布线后 WNS (最差负裕量) 为 -0.086ns ，存在轻微时序违例 (3 个 failing endpoints)，但实际上板测试中功能完全正确。关键路径位于 CIM Tile 的 MAC 链。

表 5: 时序与功耗 (ARM 控制模式)

指标	值
目标时钟频率	60 MHz
时钟约束	16.7 ns
WNS	-0.086 ns
总功耗	1.954 W
PS7	1.520 W
CIM IP	0.284 W
AXI Peripheral	0.003 W
静态功耗	0.147 W

CIM IP 本身动态功耗仅 0.284W，而 PS7 (ARM Cortex-A9 双核 + DDR 控制器) 消耗 1.520W，占总功耗的主要部分。

4.3 MLP 网络推理验证

在 ARM 控制模式下，使用 $784 \rightarrow 128 \rightarrow 10$ 两层 MLP 网络 (INT8 量化，量化后 MNIST 测试集准确率约 93%) 进行上板验证。

表 6: MLP 推理性能 (PAR_OB=1, 60MHz)

层	维度	周期数	MAC 操作数
FC1	$784 \rightarrow 128$	3,136	100,352
FC2	$128 \rightarrow 10$	146	2,048
总计		3,282	102,400

单幅推理总延迟为：

$$T_{\text{MLP}} = \frac{3282}{60 \times 10^6} = 54.7 \text{ s} \quad (10)$$

在 20 张真实 MNIST 手写数字测试中，硬件输出与 Python golden model 的 FC1 全部 128 维输出、FC2 全部 10 维 logits 逐元素完全一致 (bit-exact)，分类准确率 100% (20/20)。

4.4 LeNet-5 CNN 推理验证

LeNet-5 是一个包含 2 个卷积层、2 个最大池化层和 3 个全连接层的经典 CNN。由于 CIM 硬件本身只支持 MVM 运算，卷积层通过软件侧 im2col 变换转化为矩阵乘法后喂给硬件引擎。im2col 将卷积的滑窗操作展开为一个矩阵，使得 $\text{col_len} = C_{in} \times K \times K$ 作为 MVM 的输入维度。

表 7: LeNet-5 各层推理周期数 (PAR_OB=1, 60MHz)

层	类型	维度	平均周期数
conv1	Conv 5×5	1×28×28→6×24×24	63,360
conv2	Conv 5×5	6×12×12→16×8×8	10,112
fc3	FC	400→120	1,552
fc4	FC	120→84	876
fc5	FC	84→10	134
总计			76,034

单幅推理总延迟为：

$$T_{\text{LeNet5}} = \frac{76034}{60 \times 10^6} = 1.267 \text{ ms} \quad (11)$$

在 200 张真实 MNIST 手写数字测试中：

- 硬件分类准确率 (vs 标签)：99.5% (199/200)
- 与 Python golden model 的 bit-exact 一致率：100.0% (200/200)
- 计算错误数：0

200 张图片中唯一的”错误”分类 (img_0018, 标签 3 预测为 8) 与 Python golden model 的预测完全一致，说明这是 INT8 量化精度损失导致的，而非硬件计算错误。全部 200 张图片的全部层输出均与 Python 参考模型 bit-exact 一致，验证了硬件实现的正确性。

实际上板运行 200 张 LeNet-5 推理总 wall time 为 708.3 秒 (3.54 秒/张)，其中绝大部分时间消耗在 Python 驱动的数据加载与 im2col 计算上，硬件计算本身仅占 1.267ms/张。

4.5 加速比分析

为量化 CIM 硬件加速效果，需要将硬件推理时间与纯软件基线进行对比。

4.5.1 理论分析

CIM 硬件的核心优势在于利用存储阵列的并行性实现高通量 MAC 运算。以 FC1 层 (784→128) 为例：

- 总 MAC 操作数: $784 \times 128 = 100,352$
- 硬件周期数: 3,136
- 每周期有效 MAC 数: $100352/3136 = 32 \text{ MACs/cycle}$
- 吞吐率 (60MHz): $32 \times 60 \times 10^6 = 1.92 \text{ GMACs/s}$

而 ARM Cortex-A9 (Zynq-7020 PS, 双核 650MHz) 执行 INT8 矩阵乘法时, 即使利用 NEON SIMD 指令, 单核每周期最多执行 8 个 INT8 MAC (128 位 SIMD 寄存器 / 16 位乘积), 理论峰值吞吐率为 $8 \times 650 \times 10^6 = 5.2 \text{ GMACs/s}$ 。但这一峰值假设了完美的数据复用与零延迟访存, 实际效率受限于 cache miss、指令开销与数据搬运, 对于 784×128 这种中等规模矩阵, 实际吞吐率通常远低于峰值。

4.5.2 硬件推理时间

根据性能计数器的实测数据:

表 8: CIM 硬件推理延迟

网络	总周期数	频率	推理延迟
MLP (784→128→10)	3,282	60 MHz	54.7 s
LeNet-5 (7 层)	76,034	60 MHz	1,267 s

4.5.3 软件基线与加速比

软件基线在 PYNQ-Z2 的 ARM Cortex-A9 (650MHz) 上使用纯 NumPy INT32 矩阵运算实测。测量方法: 100 次迭代取均值 (`time.perf_counter()` 计时, 排除数据加载开销), 随机权重与硬件同维度。PYNQ 的 Python/NumPy 环境未启用 NEON SIMD 加速, 代表嵌入式 Python 的实际基线性能。

表 9: SW 基线与 HW 推理延迟对比 (ARM Cortex-A9 @ 650MHz vs CIM @ 60MHz)

网络	SW 延迟 (s)	HW 延迟 (s)	加速比
MLP (784→128→10)	1,410.8	54.7	25.8×
LeNet-5 (7 层, sequential)	26,466.2	1,267.2	20.9×
LeNet-5 (7 层, batched BLAS)	4,528.5	1,267.2	3.6×

MLP 的 SW 延迟约 1411 s 主要由 FC1 层 (784×128 , 1368 s) 主导; HW 延迟 54.7 s 来自性能计数器实测 3282 周期, 加速比约 **25.8 倍**。

LeNet-5 的比较分两种模式: **sequential 模式**与硬件相同地按输出像素逐一调用矩阵乘法 (conv1 576 次、conv2 64 次), SW 延迟约 26.5ms, 加速比约 **20.9**

倍；**batched 模式**将所有像素合并为单次矩阵乘法调用（BLAS 路径），SW 延迟约 4.5ms，加速比约 3.6 倍。Sequential 模式是更公平的对比（与硬件计算模式等价），batched 模式代表充分利用 ARM CPU 的最优 SW 实现上界。两种模式下 CIM 硬件均具有明显加速优势。

4.6 PicoRV32 验证结果

PicoRV32 模式在 PYNQ-Z2 上使用 50MHz 时钟（20ns 约束），所有时序约束均满足（WNS = +0.204ns）。

表 10: PicoRV32 模式资源占用（PYNQ-Z2）

资源类型	使用量	可用量	占用率	相对 ARM 模式增量
Slice LUT	12,714	53,200	23.90%	+1,627 (+14.7%)
Slice Register	6,272	106,400	5.89%	+887 (+16.5%)
Block RAM	44	140	31.43%	+9 (+25.7%)
DSP48E1	220	220	100.00%	+0

PicoRV32 模式相比 ARM 模式的额外开销为：1,627 个 LUT（来自 PicoRV32 核心、bridge、UART TX 等）、887 个 FF、9 个 BRAM（FW BRAM 32KB + Result BRAM 256B + AXI 互连）。DSP 使用量不变，因为 CIM IP 本身未修改。

表 11: PicoRV32 模式时序与功耗

指标	值
目标时钟频率	50 MHz
时钟约束	20.0 ns
WNS	+0.204 ns（时序收敛）
总功耗	1.895 W
PS7	1.526 W
RV32 (含 CIM)	0.217 W
AXI 互连	0.002 W
静态功耗	0.150 W

PicoRV32 模式相比 ARM 控制模式频率降低（50MHz vs 60MHz），这是因为 PicoRV32 的组合逻辑（指令解码 + ALU）引入了额外的关键路径。但 RV32 子系统的动态功耗（0.217W，含 CIM IP）反而低于 ARM 模式下 CIM IP 的 0.284W，这是因为 50MHz 的较低频率减少了动态功耗。

4.6.1 PicoRV32 功能验证

在 PYNQ-Z2 上针对 784→16→10 小型 MLP 模型（INT8 量化，FW BRAM 中约 14.5KB）进行了 200 张 MNIST 手写数字图片的端到端推理验证。

表 12: PicoRV32 推理验证结果 (784→16→10 MLP, 200 张 MNIST 图片)

指标	值
测试图片数	200
分类准确率 (vs 标签)	96.5% (193/200)
Bit-exact 一致率 (vs Python golden)	100.0% (200/200)
计算错误数	0

200 张图片的全部 10 维 logits 输出均与 Python golden model 逐元素 bit-exact 一致, 计算错误为零。7 张分类错误均源于 784→16→10 模型的 INT8 量化精度损失 (该模型在完整测试集上准确率约 90%), 与 ARM PS 控制模式下相同模型的预测结果完全一致, 验证了 RISC-V 固件控制逻辑的正确性。

4.7 两种控制模式对比

表 13: ARM 控制 vs PicoRV32 控制模式对比

指标	ARM 控制	PicoRV32 控制
时钟频率	60 MHz	50 MHz
时序收敛	WNS=-0.086ns (轻微违例)	WNS=+0.204ns (收敛)
LUT 占用	11,087 (20.84%)	12,714 (23.90%)
FF 占用	5,385 (5.06%)	6,272 (5.89%)
BRAM 占用	35 (25.00%)	44 (31.43%)
PS 依赖	需要 ARM PS 运行 Python	PS 仅用于固件加载
支持网络	任意 (受 MAX_DIM 限制)	受 FW BRAM 32KB 限制
CIM IP 修改	-	无修改
动态功耗	1.807 W	1.745 W

5 总结与展望

5.1 工作总结

本文设计并实现了一种面向边缘智能推理的低功耗 Int8 数字存算一体矩阵计算阵列架构, 在 PYNQ-Z2 (Zynq-7020) FPGA 平台上完成了完整的综合部署与板级验证。主要成果总结如下:

(1) **架构设计**: 以 16×16 CIM Tile 为原子计算单元, 设计了包含 7 级流水线的核心加速器引擎, 支持通过 CSR 在运行时配置任意层维度与量化参数。所有设计参数集中管理于 cim_pkg.sv, 实现了模块化、参数化的 RTL 框架。

(2) **工程问题解决**: 针对 Vivado BRAM 推断失败、时序违例等工程问题, 分别通过 16-bank 权重 SRAM 拆分和多级流水线插入解决, 将关键路径从 28.7ns 降低至约 16.8ns (布线后关键路径为 CIM Tile MAC 链), 在 60MHz 时序约束下完成功能验证 (WNS=-0.086ns)。

(3) **软核集成**: 成功集成 PicoRV32 RISC-V 软核替代 ARM PS 进行推理控制, CIM IP 完全不修改, 仅增加 1,627 个 LUT 和 9 个 BRAM 的开销, 以 50MHz 时序收敛。200 张 MNIST 图片验证准确率 96.5%, 全部 logits 与 ARM 路径 bit-exact 一致, 验证了 RISC-V 固件控制逻辑的正确性。

(4) **验证完备性**: 建立了“Python golden model $\rightarrow$ VCS 仿真 $\rightarrow$ PYNQ 上板”三层一致的验证流程。MLP 网络 20 张图片 100% 准确率, LeNet-5 CNN 200 张图片 99.5% 准确率 (100% bit-exact 一致)。

(5) **性能数据**: MLP 单幅推理 3,282 周期 (54.7 s @ 60MHz), LeNet-5 单幅推理 76,034 周期 (1.267ms @ 60MHz)。相对 ARM Cortex-A9 NumPy 基线, MLP 加速比约 25.8 倍, LeNet-5 加速比约 20.9 倍 (sequential 模式)。CIM IP 动态功耗仅 0.284W。

5.2 不足与展望

(1) **DSP 占用率**: 当前设计 DSP48E1 占用率达到 100%, 限制了并行度的进一步提升。后续可通过将部分乘法器映射到 LUT 来释放 DSP 资源, 或者迁移到资源更丰富的平台 (如 Kria KV260 的 Zynq UltraScale+)。

(2) **最大维度限制**: MAX_IN_DIM=784、MAX_OUT_DIM=128 的限制适用于 MNIST 尺寸网络, 但无法支持 ImageNet 级别模型。后续可通过分块加载、外部 DDR 存储或扩展到更大 BRAM 容量的平台来解决。

(3) **KV260 移植**: Kria KV260 搭载 Zynq UltraScale+ (xck26), 具有更多 LUT (234K vs 53K)、BRAM (144 vs 140 个 36Kb) 和 DSP (1248 vs 220 个), 可支持更大的 PAR_OB 和更高的 MAX_DIM。移植工作已初步开始 (board file 更换与约束文件准备), 尚未完成综合与上板验证。

(4) **bit-plane 计算模式**: 当前设计的 CIM Tile 采用直接 MAC 方式, 未实现 bit-plane (按位平面分解) 的计算模式。bit-plane 方法可以在不使用 DSP 的情况下实现 MAC, 通过按位与 + 移位累加替代乘法器, 从而大幅降低 DSP 使用量, 是后续优化的重要方向。

(5) **更复杂网络支持**: 当前系统已支持 Conv + FC + ReLU + MaxPool 组成的网络, 但尚未实现 AveragePool、BatchNorm、Softmax 等操作。这些操作可在软件侧 (Python 或 C 固件) 中实现, 硬件侧只需提供 MVM 引擎即可。

6 参考文献

- [1] REDMON J, DIVVALA S, GIRSHICK R, et al. You Only Look Once: Unified, Real-Time Object Detection[EB/OL]. 2016. <https://arxiv.org/abs/1506.02640>. arXiv: 1506.02640 [cs.CV].
- [2] LIU W, ANGUELOV D, ERHAN D, et al. SSD: Single Shot MultiBox Detector[M/OL]//Computer Vision –ECCV 2016. Springer International Pub-

- lishing, 2016: 21-37. http://dx.doi.org/10.1007/978-3-319-46448-0_2. DOI: 10.1007/978-3-319-46448-0_2.
- [3] WULF W A, MCKEE S A. Hitting the memory wall: implications of the obvious[J/OL]. SIGARCH Comput. Archit. News, 1995, 23(1): 20-24. <https://doi.org/1>. DOI: 1.
- [4] HOROWITZ M. 1.1 Computing's energy problem (and what we can do about it)[C/OL]//2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC). 2014: 10-14. DOI: 10.1109/ISSCC.2014.2.
- [5] CHEN Y H, KRISHNA T, EMER J S, et al. Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks[J/OL]. IEEE Journal of Solid-State Circuits, 2017, 52(1): 127-138. DOI: 10.1109/JSSC.2016.2616357.
- [6] SUN Z, KVATINSKY S, SI X, et al. A full spectrum of computing-in-memory technologies[J/OL]. 2023, 6(11): 823-835. DOI: 10.1038/s41928-023-01053-4.
- [7] YOSHIOKA K, ANDO S, MIYAGI S, et al. A review of SRAM-based compute-in-memory circuits[J/OL]. Japanese Journal of Applied Physics, 2024, 63(12): 120802. <http://dx.doi.org/10.35848/1347-4065/ad93e0>. DOI: 10.35848/1347-4065/ad93e0.
- [8] JACOB B, KLIGYS S, CHEN B, et al. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference[EB/OL]. 2017. <https://arxiv.org/abs/1712.05877>. arXiv: 1712.05877 [cs.LG].
- [9] YAN B, HSU J L, YU P C, et al. A 1.041-Mb/mm² 27.38-TOPS/W Signed-INT8 Dynamic-Logic-Based ADC-less SRAM Compute-in-Memory Macro in 28nm with Reconfigurable Bitwise Operation for AI and Embedded Applications[C]//2022 IEEE International Solid-State Circuits Conference (ISSCC). San Francisco, CA, USA, 2022: 188-190.
- [10] SUN Z, KVATINSKY S, SI X, et al. A full spectrum of computing-in-memory technologies[J/OL]. Nature Electronics, 2023, 6: 823-835. <https://api.semanticscholar.org/CorpusID:265164939>.
- [11] ZHANG Z, CHEN J, CHEN X, et al. From macro to microarchitecture: reviews and trends of SRAM-based compute-in-memory circuits.[J/OL]. Sci. China Inf. Sci., 2023, 66(10). <http://dblp.uni-trier.de/db/journals/chinaf/chinaf66.html#ZhangCCGWXKPHSY23>.
- [12] CHANG L, LI C, ZHANG Z, et al. Energy-efficient computing-in-memory architecture for AI processor: device, circuit, architecture perspective[J/OL].

- Science China Information Sciences, 2021, 64(6). <http://dx.doi.org/10.1007/s11432-021-3234-0>. DOI: 10.1007/s11432-021-3234-0.
- [13] SHAFIEE A, NAG A, MURALIMANO HAR N, et al. ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars [J]. ACM SIGARCH Computer Architecture News, 2016, 44(3): 14-26.
 - [14] CHI P, LI S, XU C, et al. Prime: A novel processing-in-memory architecture for neural network computation in reram-based main memory[J]. ACM SIGARCH Computer Architecture News, 2016, 44(3): 27-39.
 - [15] ANKIT A, HAJJ I E, CHALAMALASETTI S R, et al. PUMA: A programmable ultra-efficient memristor-based accelerator for machine learning inference[C]//Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems. 2019: 715-731.
 - [16] CHENG C, TIW P J, CAI Y, et al. In-memory computing with emerging nonvolatile memory devices[J]. Science China Information Sciences, 2021, 64(12): 221402.
 - [17] SESHADRI V, LEE D, MULLINS T, et al. Ambit: In-memory accelerator for bulk bitwise operations using commodity DRAM technology[C]//Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture. 2017: 273-287.
 - [18] ECKERT C, WANG X, WANG J, et al. Neural cache: Bit-serial in-cache acceleration of deep neural networks[C]//2018 ACM/IEEE 45Th annual international symposium on computer architecture (ISCA). 2018: 383-396.
 - [19] IMANI M, GUPTA S, KIM Y, et al. Floatpim: In-memory acceleration of deep neural network training with high precision[C]//Proceedings of the 46th International Symposium on Computer Architecture. 2019: 802-815.
 - [20] TU F, WANG Y, WU Z, et al. A 28nm 29.2 TFLOPS/W BF16 and 36.5 TOPS/W INT8 reconfigurable digital CIM processor with unified FP/INT pipeline and bitwise in-memory booth multiplication for cloud deep learning acceleration[C]//2022 IEEE International Solid-State Circuits Conference (ISSCC): vol. 65. 2022: 1-3.
 - [21] TU F, WANG Y, WU Z, et al. ReDCIM: Reconfigurable digital computing-in-memory processor with unified FP/INT pipeline for cloud AI acceleration [J]. IEEE Journal of Solid-State Circuits, 2022, 58(1): 243-255.
 - [22] TU F, WU Z, WANG Y, et al. TranCIM: Full-digital bitline-transpose CIM-based sparse transformer accelerator with pipeline/parallel reconfigurable modes[J]. IEEE Journal of Solid-State Circuits, 2022, 58(6): 1798-1809.

- [23] TU F, WU Z, WANG Y, et al. MulTCIM: Digital computing-in-memory-based multimodal transformer accelerator with attention-token-bit hybrid sparsity[J]. IEEE Journal of Solid-State Circuits, 2023, 59(1): 90-101.
- [24] CHOI J, WANG Z, VENKATARAMANI S, et al. Pact: Parameterized clipping activation for quantized neural networks[J]. arXiv preprint arXiv:1805.06085, 2018.
- [25] ESSER S, MCKINSTRY J L, BABLANI D, et al. Learned step size quantization[Z]. US Patent 11,823,054. 2023.
- [26] GUO X, WANG G, WANG S. Technology Developments and Applications of In-memory Computing Processors[J]. 电子与信息学报, 2023, 45(5): 1888-1898.
- [27] CHIU Y C, ZHANG Z, CHEN J J, et al. A 4-Kb 1-to-8-bit configurable 6T SRAM-based computation-in-memory unit-macro for CNN-based AI edge processors[J]. IEEE Journal of Solid-State Circuits, 2020, 55(10): 2790-2801.